```
module Document
  def print_to_screen
    prepare_cover
    format_for_screen
    print
  end

  def format_for_screen
    # ...
  end

  def print
    # ...
  end
end

class Book
  include Document
  include Printable
  # ...
end
```

Another source file creates a Book and calls print_to_screen:

```
b = Book.new
b.print_to_screen
```

According to the company's bug management application, there is a problem with this code: print_to_screen is not calling the right print method. The bug report doesn't provide anymore details.

Can you guess which version of print gets called—the one in Printable or the one in Document? Try drawing the chain of ancestors on paper. How can you quickly fix the code so print_to_screen calls the other version of print instead?

Quiz Solution

You can ask Ruby itself for the ancestors chain of Book:

```
Book.ancestors  # => [Book, Printable, Document, Object, Kernel, BasicObject]
```

If you draw this ancestors chain on your whiteboard, it will look like Figure 5, *The ancestors chain of the Book class*, on page 41.

Let's see how Ruby builds the chain. Because Book doesn't have an explicit superclass, it implicitly inherits from Object, which in turn includes Kernel and inherits from BasicObject. When Book includes Document, Ruby adds Document to Book's ancestors chain right above Book itself. Immediately after that, Book